

PADEL CV SYSTEM

Full Project Roadmap

Accuracy-First Build — Step by Step

Computer Vision · Dual-Camera Fusion · Deep-EIoU Tracking · Jetson

Hussein Ibrahim & Jeffry Abou Zeidan

AI Engineer Interns — InMobiles Holding
June 2026

Contents

1. Project Overview

InMobiles Holding operates a sports facility with a padel court equipped with Hikvision cameras. This project builds a Computer Vision system that analyzes padel matches in real time — detecting players, tracking them with stable identities, recognizing their shots, and generating post-match reports.

This roadmap is **accuracy-first** and built as a single, ordered sequence of steps. Each step is completed, run, and verified before the next one starts — so the system is always in a known-working state. The build uses a **fixed two-camera setup** running **simultaneously**, each camera covering 50–60% of the court so their views overlap around the net. That overlap is what geometrically resolves the occlusion no single-camera tracker can handle.

1.1 Goals

- Hold a **stable identity** for every player across the whole match, through occlusion and side switches
- Run **two cameras simultaneously**, each covering 50–60%, fused into one set of player IDs
- Real-time 17-joint skeleton overlay per player
- **Live top-down court view** with each player as a dot at their real court position
- Recognize padel shots: swing, smash, volley, serve, lob
- Track the ball and estimate shot speed
- Per-player session report (PDF + JSON) after each match
- 30+ FPS per stream on NVIDIA Jetson with TensorRT

1.2 Team

Name	Role
Hussein Ibrahim	AI Engineer Intern — full pipeline build, integration, deployment
Jeffrey Abou Zeidan	AI Engineer Intern — co-owner, calibration + data/model support

This build is executed step by step in one codebase; the build instructions below are written in the second person (“you”) for a single builder working through them in order.

1.3 Facility & Camera Setup

- Location: InMobiles Holding — Mkalles, Lebanon
- Court: standard padel court — 20 m × 10 m
- Cameras: **2 × Hikvision**, mounted high at opposite ends, each covering **50–60% of the court** with a shared overlap around the net
- Camera angle: overhead / high-corner (the reference still angle)
- Stream: RTSP feed per camera from the Hikvision NVR
- Edge device: NVIDIA Jetson — both streams in parallel

Why 50–60% each: the middle overlap (net area) is seen by both cameras from different angles. When two players line up on one camera, they are separated on the other. The overlap is the core accuracy mechanism — see Section 4 and build Step 7.

1.4 How to Use This Document

Sections 2–4 and 6–10 are the technical reference (what each tool is and why). **Section 5 is the build itself** — a linear list of steps. Work top to bottom: build a step, run the given command, confirm every item in its “verify it works” checklist, then move to the next step. Do not skip ahead; later steps assume earlier ones are working.

2. Technology Stack

Every choice is made for a reason. This is the reference the build steps draw on.

2.1 Player Detection & Skeleton — YOLOv11-Pose

Detects every person and extracts their 17-joint skeleton in a single pass.

Property	Detail
What it detects	Bounding box + 17 COCO keypoints per person
Speed	nano model: ~15 FPS CPU / ~45 FPS Jetson + TensorRT per stream
Why YOLOv11	Better small-object detection and occlusion handling vs v8
Library / model	ultralytics · yolo11n-pose.pt (auto-downloads)

2.2 Player Tracking — Deep-EIoU (replaces ByteTrack)

ByteTrack was replaced by Deep-EIoU. ByteTrack associates on IoU + a Kalman constant-velocity model; when two padel players overlap along a camera line and separate, it has no appearance info and guesses from box position — exactly when ID swaps happen. Deep-EIoU is built for sports: it drops the constant-velocity assumption and combines Iterative Scale-up ExpansionIoU with deep appearance features.

Property	Detail
What it does	Stable per-player IDs, combining motion (ExpansionIoU) + appearance (ReID)
Why Deep-EIoU	Built for sports: irregular motion + similar jerseys + heavy occlusion
Benchmark	SOTA on SportsMOT (77.2 HOTA) and SoccerNet (85.4 HOTA), no extra compute vs appearance-motion trackers
ReID backbone	OSNet — lightweight, discriminative for look-alike athletes
Integration	boxmot (clean pip interface) or the Deep-EIoU repo; fed our YOLOv11 detections

The accuracy stack (layered): Deep-EIoU base + pose-keypoint-aware association + OSNet ReID fine-tuned on padel + jersey color as a hard team gate + court-homography zone logic + dual-camera fusion (+ optional global tracklet association for near-offline correction).

2.3 Identity & ReID — OSNet + Jersey Color + Homography

- **OSNet embeddings** — match a player after occlusion by appearance, not box position; fine-tuned on padel footage
- **Jersey color (HSV)** — permanent identity anchor and hard team-side gate; works on a single stream

- **Court homography** — pixel → real court coords; same position across cameras = same ID

2.4 Multi-Camera Fusion & Live Court View

Both cameras run at once; each produces tracked players projected through its homography into one shared 20 m × 10 m court frame. In that frame, detections from both cameras merge: a player in the overlap zone seen by both maps to one position → one dot → one global ID.

- **Top-down minimap (recommended)** — a canvas drawn as the real padel court (sidelines, service lines, center service line, net); player dots + ball from fused coordinates
- **Dual camera display** — both feeds side by side with overlays, minimap panel beneath

2.5 Action, Ball, Video, Reports

- **Actions:** rule-based → LSTM → ST-GCN accuracy ladder (Section 6)
- **Ball:** background subtraction → custom YOLO → TrackNet (Section 7)
- **Video:** OpenCV for I/O, undistort, homography, overlays, minimap
- **Reports:** JSON (machine) + PDF via fpdf2 (human), auto-generated per session

2.6 Library List

Library	Install	Purpose
ultralytics	pip install ultralytics	YOLOv11-Pose detection + skeleton
boxmot	pip install boxmot	Strong ReID trackers + OSNet, wired to YOLO detections
Deep-EIoU	clone hsiangwei0903/Deep-EIoU	SOTA sports tracker (primary target)
torchreid	pip install torchreid	OSNet ReID — fine-tune on padel
opencv-python	pip install opencv-python	Video I/O, undistort, homography, minimap
numpy	pip install numpy	Angles, distances, arrays
fpdf2	pip install fpdf2	PDF reports
torch	pip install torch	YOLO + ReID + action models
flask	pip install flask	Web dashboard (optional)

3. System Architecture

3.1 Dual-Camera Simultaneous Pipeline

One capture thread per camera runs the per-camera stages in parallel; a coordinator thread fuses them into global IDs before downstream analysis.

Per-camera (both cameras at once):

Step	Component	Output
1	Video Capture	Raw frames
2	Undistort	Lens-corrected frame
3	YOLOv11-Pose	bbbox + 17 keypoints + conf
4	Court ROI Filter	Only on-court detections
5	Deep-EIoU Track	Per-camera track IDs
6	Jersey ReID	Jersey color per player
7	Homography	Real court position (x_cm, y_cm)

Fusion + downstream (coordinator):

Step	Component	Output
8	Cross-Camera Fusion	Merged global player IDs (overlap resolved)
9	Enrollment Match	Player name
10	Action Detector	Action + confidence
11	Ball Detector	Ball position + speed
12	Visualizer	Dual display + live minimap
13	Report Logger / Session Report	JSON + PDF

3.2 File Structure

File / Folder	Purpose
main.py	Entry point — reads config, launches camera threads + coordinator
config.json	Two camera sources, court polygons, homography paths, settings
players.json	Enrolled names + jersey colors
core/pipeline.py	Per-camera worker loop
core/fusion.py	Cross-camera merge → global IDs

File / Folder	Purpose
core/detector.py	YOLOv11-Pose wrapper
core/tracker.py	Deep-EIoU wrapper (motion + OSNet + keypoints)
core/actions.py	Action detection (rules + AI)
core/ball.py	Ball detection
core/report.py	Session logging + JSON/PDF
utils/jersey.py	Jersey color (HSV)
utils/skeleton.py	COCO-17 drawing
utils/homography.py	Calibration + pixel→court mapping
utils/court_view.py	Live top-down minimap
utils/reid.py	OSNet embeddings + matching
utils/enrollment.py	Player enrollment
utils/clips.py	Auto-clip export
utils/heatmap.py	Position heatmap
utils/colors.py	Color palettes
models/ · data/ · output/ · calibration/	Models, training data, outputs, calibration files

3.3 Config — Controls Everything

Key	Description
cameras	Exactly 2 configs, both active, processed in parallel
cameras[].source	RTSP URL (or video file during development)
cameras[].coverage	Which 50–60% it covers: 'left' / 'right'
cameras[].homography	Path to that camera's H matrix (.npy)
cameras[].court_polygon	4 pixel corners in this camera's view
overlap_zone	Court x-range seen by both cameras — used by fusion
court_real_size	[20.0, 10.0] meters — shared coordinate frame
net_x	Net position in court coords — team-side assignment
tracker / reid_weights	'deep_eiou' + OSNet weights fine-tuned on padel
model	yolo11n-pose.pt or yolo11n-pose.engine (TensorRT)

4. Dual-Camera System — Fixed 2 Cameras, 50–60% Each

Two fixed cameras run at the same time, each covering 50–60% of the court so their views overlap around the net. The overlap resolves occlusion a single camera cannot.

4.1 Coverage & Placement

Camera	Covers	Position
Camera 1	~50–60% — near half + middle overlap	Back corner / high end, angled to far baseline
Camera 2	~50–60% — far half + middle overlap	Opposite back corner / high end

Aim both cameras so the net region sits inside both frames — that is where most occlusions happen in padel.

4.2 Setup (per camera, one time)

- **Install:** mount, connect to NVR, note RTSP URL, add to config with coverage label
- **Calibrate:** 9×6 checkerboard, ~20 photos, `cv2.calibrateCamera()` → `calibration/camN.npy`, apply `cv2.undistort()` each frame
- **Homography:** 4 court points visible in that camera, measure real coords (20 m × 10 m), `cv2.findHomography()` → `calibration/camN_H.npy`

4.3 Live Court View

- Project each player's foot point (ankle midpoint) through homography into court coords → dot on a court canvas drawn as the **real padel court** (sidelines, service lines, center service line, net), updated every frame
- Ball drawn on the same minimap; dual feeds shown side by side with the minimap beneath
- Fusion is visible here: one occlusion event shows as two cleanly separated dots on the minimap

5. The Build — Step by Step

The entire system runs on **two cameras**. Every step processes both feeds in parallel — there is no single-camera mode. During development the two feeds come from recorded clips set in `config.json`; in production they are the two RTSP streams. Both cameras must be mounted in their final 50–60% overlap positions and calibrated before Step 5, because that step locks both cameras into **one shared court coordinate frame** — which is what makes fusion (Step 6) possible. Work top to bottom: build, run, verify, then proceed. **Step 6 is the accuracy gate**. Action recognition (Steps 7–8) is a **trained neural network** — there is no rule-based detection.

Step 1 — Dual-Camera Capture & Detection

Goal: Capture both camera feeds in parallel and run YOLOv11-Pose with skeletons on each.

Create / edit:

- Project folders + Python venv; install the library list (Section 2.6)
- `config.json` with **two** camera sources (recorded clips now, RTSP later)
- `main.py` + `core/pipeline.py` — one capture/inference thread per camera, run in parallel
- `core/detector.py` — YOLOv11-Pose wrapper; `utils/skeleton.py` — COCO-17 + FPS overlay

Run: `python main.py --show`

Verify it works:

- Two windows open (one per camera), both playing in parallel without crashing
- Every player in each feed has a bounding box and a 17-joint skeleton
- An FPS counter is shown for each stream

Unlocks: A two-camera detection foundation.

Step 2 — Court Polygon — Auto-Detect, Editable + Foot-Point Filter

Goal: Have the system automatically draw the court polygon per camera, let you edit it, and keep only players whose FEET are inside it.

Create / edit:

- Auto-detect: threshold the white court lines in HSV + Hough line transform (or the largest playing-surface contour) to propose the 4 court corners per camera
- Editable overlay: draw the proposed polygon; click-drag corners to correct it; save to `cameras[].court_polygon` in `config.json`
- Foot-point filter: a player is on court if the midpoint of their two ankle keypoints (fallback: `bbox` bottom-center) is inside the polygon (`cv2.pointPolygonTest`)

Run: `python main.py --setup-court (auto-detect + edit per camera) then python main.py --show`

Verify it works:

- The polygon is auto-drawn over the real court in each camera's view as a starting guess

- You can drag any corner to fix it and the change is saved to config
- Only players whose **feet** fall inside the polygon are kept — someone behind the glass or reaching in from outside is excluded

Unlocks: Clean, foot-accurate on-court detections in both feeds.

Step 3 — Player Tracking — Deep-EIoU (both cameras)

Goal: Stable per-camera IDs using Deep-EIoU on each feed.

Create / edit:

- `core/tracker.py` — Deep-EIoU (boxmot or the repo) fed YOLOv11 detections + keypoints; one tracker instance per camera
- Load OSNet ReID weights; draw a unique color + ID per player (`utils/colors.py`)
- (Optional, for top accuracy) fine-tune OSNet on padel player crops — see Section 9

Run: `python main.py --show`

Verify it works:

- Each feed shows stable IDs that do not flicker frame to frame
- IDs are per-camera at this stage (the two cameras number independently) — they get unified in Step 6
- Note any net-exchange swaps in each feed — fixed by fusion in Step 6

Unlocks: Per-camera identities to fuse.

Step 4 — Jersey Color + Enrollment (Shared Identity Anchor)

Goal: A permanent identity anchor shared across both cameras.

Create / edit:

- `utils/jersey.py` — dominant torso color in HSV, classified to a named color
- `utils/enrollment.py` — capture name + jersey → `players.json` (one registry for the whole system)
- ID-swap re-check by jersey color, applied in each feed

Run: `python main.py --enroll` then `python main.py --show`

Verify it works:

- Players are labeled by their real name in both feeds
- Jersey color stays stable; when a tracker ID swaps, the jersey re-check restores the correct name

Unlocks: A shared identity signal both cameras can agree on.

Step 5 — Dual-Camera Homography — Real Padel Court Frame + Live Minimap

Goal: Calibrate BOTH cameras into ONE shared frame shaped like the real padel court (same lines), and render a minimap that reproduces the actual court markings.

Create / edit:

- Define the real padel court template in court coordinates: outer 20 m × 10 m, net line across the center, two service lines 3 m from the net, and the center service line between them
- `utils/homography.py` — for EACH camera, findHomography from the **same real court line intersections** (corners, service-line $\cap$ sideline, center-line ends) into that shared template — both cameras use the same reference points
- `utils/court_view.py` — render the full padel court template (all lines) and overlay player dots

Run: `python main.py --show --minimap`

Verify it works:

- The minimap shows the real padel court lines (sidelines, service lines, center service line, net) — not a blank rectangle
- A player standing on a known court line lands exactly on that line on the minimap, from either camera (proves a shared, true-to-court frame)
- Dots track correctly as players move

Both homographies must use the same real court reference points / origin. If camera 1 and camera 2 are calibrated to different references, a player will appear at two different minimap spots and fusion (Step 6) cannot match them. Verify the on-the-line test above before continuing.

Unlocks: A true-to-court shared frame that fusion needs.

Step 6 — Cross-Camera Fusion — ACCURACY GATE

Goal: Merge both cameras' tracks into ONE set of global player IDs and resolve occlusion.

Create / edit:

- `core/fusion.py` — in the overlap zone, match by shared court position + jersey color to collapse two camera-local tracks into one global ID
- Shared global player registry protected by `threading.Lock`

Run: `python main.py --show --minimap`

Verify it works:

- A net-crossing that swapped IDs in a single feed (Step 3) now holds, because the other camera separates the players
- The fused minimap shows exactly one dot per player, with the correct name, through occlusion

This is the accuracy milestone. Do not proceed until a real net exchange holds global IDs on the fused minimap. If it fails, fix it here (shared-frame homography accuracy, overlap matching, or jersey gate) before building actions, ball, or reports.

Unlocks: Trustworthy global identities — everything downstream is now safe to build.

Step 7 — Action Data Collection & Labeling

Goal: Collect and label keypoint-sequence data to train the action model. No rule-based detection is used — this is the dataset for the neural network.

Create / edit:

- Keypoint auto-saver: for each tracked player, save 30-frame windows of shape (30, 17, 3) to data/unlabeled/ with the matching short clip
- Run real matches to accumulate clips — target 300+ per action class (this runs in the background across many sessions)
- Label in CVAT (app.cvat.ai) with the 6 action classes (the table in Section 6 defines them); export and move to data/labeled/{action}/

Run: `python main.py --collect` (saves windows while the pipeline runs)

Verify it works:

- data/unlabeled/ fills with (30, 17, 3) windows + reference clips as matches play
- After labeling, data/labeled/ has clips sorted into the 6 action folders

Data collection runs across multiple sessions — it is the long pole. You can build Steps 9–10 (ball, reports) while clips accumulate; come back to train once you have enough per class.

Unlocks: A labeled dataset to train the action model on.

Step 8 — Train & Integrate the Action Model (Neural Network)

Goal: Train a neural network for action recognition and run it live — this IS the action detector, no rules.

Create / edit:

- Normalize keypoints: hip center → (0,0), scale by shoulder-to-hip distance
- Train an LSTM (PyTorch, 2-layer, hidden 128, dropout 0.3, 50–100 epochs) → models/padel_lstm.pt; upgrade to ST-GCN (MMAAction2) for production accuracy
- Integrate: core/actions.py loads the trained model; each frame feeds the normalized window → action + confidence; show confidence color (green > 80%, orange 60–80%, red < 60%)

Run: train script, then `python main.py --show --minimap`

Verify it works:

- Per-class action accuracy ≥ ~85% on a held-out split (LSTM), higher with ST-GCN
- Live shots are labeled by the trained model with a confidence color; labels are stable, not flickering

Unlocks: Trained shot recognition feeding reports and clips.

Step 9 — Ball Tracking

Goal: Detect and track the ball; estimate shot speed.

Create / edit:

- `core/ball.py` — start with MOG2 background subtraction + area + circularity for quick bring-up (Section 7)
- For accuracy, train a custom YOLOv11 ball detector, then TrackNet for production (Sections 7, 9)
- Speed from $\text{px/frame} \times \text{court scale} \rightarrow \text{km/h}$

Run: `python main.py --show --minimap`

Verify it works:

- The ball is marked with a circle most of the time it is visible; a rough km/h speed is shown
- The trained detector (when ready) tracks through motion blur better than MOG2

Unlocks: Shot-speed and rally features.

Step 10 — Reports, Heatmap & Clips

Goal: Produce per-session outputs automatically — this completes a working v1.

Create / edit:

- `core/report.py` — JSON + PDF (fpdf2) at session end (Section 8)
- `utils/heatmap.py` — per-player position heatmap on the real court; `utils/clips.py` — rolling buffer auto-clips

Run: `python main.py --save-report`

Verify it works:

- `output/` contains a JSON file and a PDF after the session finishes
- The PDF has per-player stats, a timeline, and an embedded court heatmap
- Clips are saved for high-confidence smashes / serves

Unlocks: A complete working version of the dual-camera system.

Step 11 — Jetson Deployment

Goal: Run both live RTSP streams at 30+ FPS on the Jetson.

Create / edit:

- Export YOLO + OSNet + the action model to TensorRT **on the Jetson** (.engine files)
- Switch config sources to the cameras' RTSP URLs; use a GStreamer decode pipeline (Section 10)

Run: `python main.py --show --minimap` (on the Jetson, live RTSP)

Verify it works:

- 30+ FPS per stream; latency under ~200 ms court-to-display
- Runs stable through a full match with fusion + minimap live

Unlocks: Production system at InMobiles.

6. Action Detection (Reference)

Action	Key Indicator
Swing	Wrist moving fast, elbow bending
Smash	Wrist above head, arm extended, fast motion
Volley	Wrist at shoulder height, compact motion
Serve	Wrist raised high, forward motion
Lob	Wrist raised, slow arc motion
Idle	No significant wrist movement

6.1 Trained Model (no rule-based detection)

Action recognition is a **neural network trained on labeled keypoint sequences** — there are no rule-based thresholds. The table above defines the 6 classes and what each looks like; it is used as the labeling guide, not as runtime logic.

- **Input:** 30-frame × 17-keypoint windows, normalized (hip center → 0,0; scale by shoulder-to-hip distance) so the model learns body movement, not court position
- **LSTM** (baseline, 88–92%): 2-layer, hidden 128, dropout 0.3, softmax over 6 classes
- **ST-GCN** (production, 94–97%): keypoints as a spatio-temporal graph; needs 500+ samples/class; MMAAction2
- Built in Steps 7–8: collect + label data, then train + integrate

7. Ball Tracking (Reference)

- **Level 1 — MOG2** background subtraction + area (20–300 px) + circularity filter
- **Level 2 — Custom YOLOv11** trained on 500–1000 labeled ball frames (Roboflow)
- **Level 3 — TrackNet** for fast small balls; 3 frames → heatmap; fine-tune on padel

Enables	How
Shot speed (km/h)	Velocity px/frame × court scale
Bounce detection	Sudden y-position spike on floor contact
Shot confirmation	Smash = ball near wrist + moving fast
Ball heatmap	Aggregate ball positions → court usage

8. Additional Features (Reference)

- **Form error alerts** — compare joint angles to ideal ranges per shot; 2-second on-screen warning; logged per player

- **Match timeline** — every shot logged (time, player, action, confidence, position); rendered in the PDF
- **Auto-clip export** — rolling 3-second buffer; trigger on smash/serve/lob > 80%
- **Position heatmap** — fused court positions per player; PNG embedded in the PDF
- **Session comparison** — diff two JSON reports for week-over-week progress
- **Web dashboard (optional)** — Flask streams the dual view + minimap over facility WiFi

9. Data Collection & Training (Reference)

- Auto-save 30-frame keypoint windows (30, 17, 3) to data/unlabeled/; target 300+ clips/class
- Label in CVAT (app.cvat.ai) with 6 action labels; export and match to .npy files
- Normalize: hip center → (0,0), scale by shoulder-to-hip distance
- Train LSTM (PyTorch, 50–100 epochs, Adam, cross-entropy, 80/20 split)
- Fine-tune OSNet ReID on padel player crops (torchreid)
- Upgrade to ST-GCN at 500+ samples/class (MMAAction2): ~88% → ~95%

Minimum 200 labeled clips/class (1200 total). Target 500+/class (3000 total) for ST-GCN.

10. Production Deployment on Jetson (Reference)

- **TensorRT**: export yolo11n-pose.pt and OSNet on the Jetson → .engine files (hardware-specific)
- **RTSP**: rtsp://admin:password@192.168.1.X/Streaming/Channels/101 in config; GStreamer hardware decode
- **Threading**: one thread per camera + a coordinator thread; shared registry behind a Lock

Hardware / Config	FPS / stream	Notes
Laptop CPU	3–6	Dev only
Laptop GPU (CUDA)	20–30	Testing / demos
Jetson Orin, TensorRT	35–45	Target — production
Jetson Orin, TensorRT + skip ×2	50+	Max performance

11. Known Challenges & Solutions

Challenge	Solution
Ghost detections (chairs, spectators)	Auto court polygon + foot-point filter — Step 2
Player ID swap when two players cross	Deep-EIoU + OSNet + jersey gate + dual-camera fusion — Steps 3, 4, 6
Two players on the same camera line	Second camera at the opposite angle separates them — Step 6
Swing vs volley confusion	Trained LSTM / ST-GCN on labeled keypoint sequences — Steps 7–8
Ball too small and fast	MOG2 bring-up → custom YOLO → TrackNet — Step 9
ReID weaker than benchmarks suggest	Fine-tune OSNet on padel frames — Step 3 / Section 9
Real-time lag (×2 streams)	TensorRT + GStreamer + per-stream threading — Step 11
Players switch sides — wrong team	Identity = ReID + jersey; team = court side
JSON report fails on numpy types	Custom NumpyEncoder in report.py

12. Summary — What Each Step Delivers

Capability	Technology	Step
Player detection + skeleton (2 cams)	YOLOv11-Pose	1
Auto court polygon + foot filter	Line detection + editable polygon	2
High-accuracy stable IDs	Deep-EIoU + OSNet ReID	3
Jersey color + name identity	HSV + enrollment	4
Real padel court frame + minimap (2 cams)	Dual homography	5
Cross-camera fusion → global IDs	Position + jersey match	6
Action training data (labeled)	Keypoint sequences + CVAT	7
Action recognition (trained NN)	LSTM → ST-GCN	8
Ball detection + speed	MOG2 → YOLO → TrackNet	9
Reports, heatmap, auto-clips	fpdf2 + json	10
30–45 FPS per stream on Jetson	TensorRT + RTSP	11